JAVA DSA • COMPLETE INTERVIEW PREPARATION

BINARY SEARCH
ULTIMATE DEEP-DIVE HANDBOOK

Basic to Advanced • All Major Patterns • Formulas • Dry Runs • Java Implementations • Interview Problems

Is handbook ka promise: sirf binary search ka ek template nahi. Tum sorted arrays se lekar rotated arrays, answer-space search, matrices, two sorted arrays, floating-point search aur advanced interview patterns tak systematically seekhoge.

Language: Java, simple Hinglish explanations. Code reusable interview-style methods mein diya gaya hai. Har algorithm ke saath idea, invariant, implementation, complexity aur common mistakes cover kiye gaye hain.

Learning path	Coverage
Foundation	Sorted data, search space, mid formula, loop invariant, iterative & recursive
Classic patterns	First/last occurrence, lower/upper bound, count, insertion position, range
Modified arrays	Rotated, duplicates, peak, bitonic, nearly sorted, infinite array
Binary search on answer	Minimum feasible answer, maximum feasible answer, monotonic predicates
Advanced	2D matrix, kth element, median of two arrays, square root, floating point
Interview mastery	Recognition clues, dry runs, complexity, edge cases, 50+ practice prompts

Goal: question ko dekhkar search space aur monotonic condition khud identify karna.

How to study this handbook

Order follow karo: pehle sorted array par exact search, phir boundaries, phir modified arrays, phir binary search on answer. Har code ko IDE mein khud type karo. Sirf output dekhna learning nahi hai—har iteration mein low, high, mid aur decision explain karo.

Pass	Action	Checkpoint
1 — Concept	Idea, diagrams, formulas aur invariant padho.	Search space kya hai, bata sako.
2 — Code	Template ko bina dekhe Java mein likho.	Off-by-one error na ho.
3 — Dry run	Har iteration ka low, mid, high likho.	Pointer movement justify kar sako.
4 — Variation	Same pattern ke 3–5 questions karo.	Pattern identify kar sako.
5 — Complexity	Time, auxiliary space aur output space alag batao.	O(log n) kyun hai, prove kar sako.

Complete table of contents

- 1. Binary Search ka mental model and prerequisites
- 2. Search space, mid formulas and loop invariants
- 3. Iterative and recursive binary search
- 4. Lower Bound, Upper Bound and insertion position
- 5. First occurrence, last occurrence, frequency and range
- 6. Floor, ceil, predecessor and successor
- 7. Rotated sorted arrays—with and without duplicates
- 8. Minimum in rotated array and rotation count
- 9. Peak element, mountain/bitonic arrays
- 10. Nearly sorted arrays and infinite sorted arrays
- 11. Binary Search on Answer: feasibility and monotonicity
- 12. Aggressive Cows, Allocate Books, Ship Packages
- 13. Koko, bouquets, divisor and other answer-space problems
- 14. Square root, nth root and floating-point search
- 15. 2D matrix search and row/column sorted matrices
- 16. Median and kth element of two sorted arrays
- 17. Exponential, ternary and interpolation search overview
- 18. Java implementation toolkit and debugging checklist
- 19. Complexity analysis and proof techniques
- 20. Interview problem bank, hints and revision sheet

Important: Binary Search sirf array mein number dhoondhne ka naam nahi. Jab answer space ordered ho aur feasibility monotonic ho, tab bhi binary search lag sakti hai.

1. Binary Search ka mental model

Binary Search ek searching technique hai jo har step mein current search space ka lagbhag aadha hissa discard karti hai. Classic binary search ke liye data sorted hona chahiye. Lekin broader form mein sorted answer space ya monotonic predicate bhi kaafi hota hai.

Linear Search vs Binary Search

Feature	Linear Search	Binary Search
Requirement	Sorted hona zaroori nahi	Classic form mein sorted data
Per step	Ek element check	Middle element check
Worst time	O(n)	O(log n)
Extra space	O(1) iterative	O(1) iterative
Best use	Small/unsorted data	Large ordered search space

Visual example

Array: [3, 7, 11, 18, 23, 31, 42, 56, 69], target = 31. Middle index 4 par 23 hai. 31 bada hai, isliye left half discard. Remaining indices 5..8: middle index 6 par 42. Target chhota hai, right side discard. Index 5 par 31 mil gaya.

Step	low	high	mid	a[mid]	Decision
1	0	8	4	23	target > 23, low = 5
2	5	8	6	42	target < 42, high = 5
3	5	5	5	31	Found

Core idea: Har comparison ke baad aisa half discard karo jisme answer ho hi nahi sakta. Agar discard karne ka reason prove nahi kar sakte, binary search condition galat ho sakti hai.

Binary Search ke 3 pillars

- Ordered search space: elements ya possible answers ka order meaningful ho.
- Decision rule: mid dekhkar pata chale ki answer kis side ho sakta hai.
- Progress: har iteration search interval ko strictly chhota kare; infinite loop na ho.

Kab binary search nahi lagani chahiye?

- Unsorted array mein exact search ke liye direct binary search valid nahi.
- Agar condition non-monotonic hai aur middle ke basis par ek side discard nahi kar sakte, standard binary search fit nahi.
- Agar array bahut chhota hai, linear search simpler ho sakti hai; asymptotic complexity hamesha practical speed nahi batati.

2. Search space, formulas and invariants

Inclusive interval: [low, high]

Classic implementation mein low aur high dono valid candidate indices hote hain. Continue condition `low <= high` hoti hai. Mid check ke baad ek index discard karna hota hai.

Safe middle formula

```
int mid = low + (high - low) / 2;
```

`(low + high) / 2` integer overflow kar sakta hai jab indices bahut bade hon. Safe formula `low + (high-low)/2` use karo.

Pointer update

Comparison	Update	Reason
a[mid] == target	return mid	Exact match mil gaya.
a[mid] < target	low = mid + 1	Sorted order mein left + mid sab chhote.
a[mid] > target	high = mid - 1	Sorted order mein mid + right sab bade.

Loop invariant kya hota hai?

Loop invariant ek statement hai jo har iteration ke start par true rehna chahiye. Exact search ke liye invariant: agar target array mein exist karta hai, toh woh current interval [low, high] ke andar hi hai. Har update discarded half ko safely remove karta hai.

Inclusive vs half-open interval

Style	Initial range	Loop condition	Typical use
Inclusive	low=0, high=n-1	low <= high	Exact search, common templates
Half-open	low=0, high=n	low < high	Lower bound, first true predicate

Dono correct hain. Ek algorithm ke interval rules ko doosre ke pointer updates ke saath mix mat karo.

Binary search iterations ka formula

N elements ko repeatedly half karne par remaining size approx N / 2^k hota hai. Jab size 1 ke aas-paas ho: N / 2^k = 1, so 2^k = N and k = $\log_2 N$. Isliye time O(log n).

n	Approx max halving steps
8	3
16	4
1,024	10
1,000,000	20
1,000,000,000	30

Interview explanation: Har iteration mein search space half hota hai, isliye T(n)=T(n/2)+O(1), which solves to O(log n).

3. Standard Binary Search — iterative

Exact target ka index return karo; target na mile toh -1. Duplicate values hon toh ye implementation koi bhi matching index return kar sakti hai, necessarily first ya last nahi.

Complete Java implementation

```
import java.util.*;

class BinarySearchBasics {
    static int binarySearch(int[] a, int target) {
        int low = 0;
        int high = a.length - 1;

        while (low <= high) {
            int mid = low + (high - low) / 2;

            if (a[mid] == target) return mid;
            if (a[mid] < target) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
        return -1;
    }

    public static void main(String[] args) {
        int[] a = {2, 5, 8, 12, 16, 23, 38};
        System.out.println(binarySearch(a, 16)); // 4
        System.out.println(binarySearch(a, 7));  // -1
    }
}
```

Dry run: [2, 5, 8, 12, 16, 23, 38], target = 23

Iteration	low	high	mid	value	Action
1	0	6	3	12	low = 4
2	4	6	5	23	return 5

Complexity

- Time: best O(1) if first mid matches; average/worst O(log n).
- Auxiliary space: O(1) for iterative implementation.
- Input space: array already supplied, not counted as algorithm-created space.
- Output space: O(1), one integer result.

Important edge cases

- Empty array: high = -1, loop never executes, returns -1.
- One element: target match or no match.
- Target smaller than all / larger than all.
- Duplicates: exact search returns any one match.
- Integer extremes: safe mid formula.

Recursive Binary Search

Recursive version har call mein half interval par call karta hai. Base case low > high hai. Recursion call stack O(log n) frames tak ja sakta hai.

Recursive Java implementation

```
static int binarySearchRecursive(int[] a, int target,
                                 int low, int high) {
    if (low > high) return -1;

    int mid = low + (high - low) / 2;

    if (a[mid] == target) return mid;
    if (a[mid] < target) {
        return binarySearchRecursive(a, target, mid + 1, high);
    }
    return binarySearchRecursive(a, target, low, mid - 1);
}

// Call:
int index = binarySearchRecursive(a, target, 0, a.length - 1);
```

Complexity: Time O(log n), auxiliary space O(log n) due to recursion stack. Iterative is usually preferred in Java when constant extra space matters.

4. Lower Bound and Upper Bound

Ye binary search ke sabse important interview concepts hain. Duplicates, insertion position, first occurrence aur count wale problems inhi se solve hote hain.

Lower Bound

Definition: first index i jahan `a[i] >= target`. Agar koi aisa element nahi, answer n (array length).

Lower bound — half-open interval

```
static int lowerBound(int[] a, int target) {
    int low = 0, high = a.length; // [low, high)

    while (low < high) {
        int mid = low + (high - low) / 2;

        if (a[mid] < target) {
            low = mid + 1;
        } else {
            high = mid;
        }
    }
    return low;
}
```

Upper Bound

Definition: first index i jahan `a[i] > target`. Agar aisa element nahi, answer n.

Upper bound — half-open interval

```
static int upperBound(int[] a, int target) {
    int low = 0, high = a.length;

    while (low < high) {
        int mid = low + (high - low) / 2;

        if (a[mid] <= target) {
            low = mid + 1;
        } else {
            high = mid;
        }
    }
    return low;
}
```

Array	target	lowerBound	upperBound	Meaning
[1,2,2,2,4,7]	2	1	4	2s occupy indices 1..3
[1,2,2,2,4,7]	3	4	4	3 absent; insertion index 4
[1,2,2,2,4,7]	9	6	6	Insert at end
[1,2,2,2,4,7]	0	0	0	Insert at start

Count occurrences

Sorted array mein target count = upperBound - lowerBound. Isse O(log n) time mein count milta hai.

Count occurrences

```
static int countOccurrences(int[] a, int target) {
    return upperBound(a, target) - lowerBound(a, target);
}
```

Search insert position

Insert position woh first index hai jahan value >= target. Exactly lower bound.

Search insert position

```
static int searchInsert(int[] a, int target) {
    return lowerBound(a, target);
}
```

Yaad rakhne ka shortcut: Lower bound mein `< target` par right jao. Upper bound mein `<= target` par right jao. Bas ek equality ka difference hai.

Lower/Upper Bound dry run

a=[1,2,2,2,4,7], target=2. Lower bound: mid values 2 par bhi high=mid, kyunki pehla 2 chahiye. Upper bound: 2 par low=mid+1, kyunki 2 ke baad ka first element chahiye.

5. First occurrence, last occurrence and range

Sorted array with duplicates mein first/last occurrence alag se binary search kar sakte ho, ya lower/upper bound use kar sakte ho.

First and last occurrence

```
static int firstOccurrence(int[] a, int x) {
    int low = 0, high = a.length - 1, ans = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == x) {
            ans = mid;
            high = mid - 1; // keep searching left
        } else if (a[mid] < x) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return ans;
}

static int lastOccurrence(int[] a, int x) {
    int low = 0, high = a.length - 1, ans = -1;
    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == x) {
            ans = mid;
            low = mid + 1; // keep searching right
        } else if (a[mid] < x) {
            low = mid + 1;
        } else {
            high = mid - 1;
        }
    }
    return ans;
}
```

Search range: return [first, last]

LeetCode-style range solution

```
static int[] searchRange(int[] a, int target) {
    int first = firstOccurrence(a, target);
    if (first == -1) return new int[]{-1, -1};

    int last = lastOccurrence(a, target);
    return new int[]{first, last};
}
```

Dry run

a=[5,7,7,8,8,8,10], target=8. First occurrence = 3; last occurrence = 5; count = 5-3+1 = 3.

Operation	Result	Complexity
First occurrence	3	O(log n)
Last occurrence	5	O(log n)
Frequency	3	O(log n)
Range	[3,5]	O(log n)

Floor and Ceil

Floor: greatest element <= x. Ceil: smallest element >= x. Sorted array mein lower bound se ceil index milta hai; floor lowerBound-1 ho sakta hai, lekin exact match par floor target itself hai.

Floor and ceil values

```
static Integer ceilValue(int[] a, int x) {
    int i = lowerBound(a, x);
    return i == a.length ? null : a[i];
}

static Integer floorValue(int[] a, int x) {
    int i = upperBound(a, x) - 1;
    return i < 0 ? null : a[i];
}
```

Null return ki jagah -1 ya OptionalInt bhi use kar sakte ho; contract problem ke mutabik choose karo.

Predecessor and successor

- Strict predecessor: greatest value < x = lowerBound(x)-1.
- Strict successor: smallest value > x = upperBound(x).
- Duplicates ke case mein strict vs non-strict requirement carefully read karo.

6. Rotated Sorted Array

A sorted array ko kisi pivot par rotate kiya gaya hai. Example original [1,2,3,4,5,6,7], rotate → [4,5,6,7,1,2,3]. Entire array sorted nahi, lekin har iteration mein at least one half sorted hota hai (distinct values ke standard version mein).

Search in rotated sorted array — distinct values

Java solution — O(log n)

```
static int searchRotated(int[] a, int target) {
    int low = 0, high = a.length - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == target) return mid;

        // Left half is sorted.
        if (a[low] <= a[mid]) {
            if (a[low] <= target && target < a[mid]) {
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }
        // Otherwise right half is sorted.
        else {
            if (a[mid] < target && target <= a[high]) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
    }
    return -1;
}
```

Logic ko samjho

- Agar a[low] <= a[mid], left half sorted hai.
- Check karo target sorted left interval [a[low], a[mid]) mein hai? Haan toh high=mid-1; warna right side.
- Agar left half sorted nahi, right half sorted hoga. Target (a[mid],a[high]] mein hai toh right jao.
- Mid equal target check pehle karo.

Search with duplicates

Duplicates ke saath [1,0,1,1,1] jaise case mein a[low], a[mid], a[high] equal ho sakte hain. Isse sorted half identify nahi hota. Ambiguous ends ko trim karna padta hai; worst case O(n).

Rotated search with duplicates

```
static boolean searchRotatedDuplicates(int[] a, int target) {
    int low = 0, high = a.length - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;
        if (a[mid] == target) return true;

        if (a[low] == a[mid] && a[mid] == a[high]) {
            low++;
            high--;
        } else if (a[low] <= a[mid]) {
            if (a[low] <= target && target < a[mid]) high = mid - 1;
            else low = mid + 1;
        } else {
            if (a[mid] < target && target <= a[high]) low = mid + 1;
            else high = mid - 1;
        }
    }
    return false;
}
```

Minimum element in rotated sorted array

Minimum — distinct values

```
static int findMinRotated(int[] a) {
    int low = 0, high = a.length - 1;

    while (low < high) {
        int mid = low + (high - low) / 2;
        if (a[mid] > a[high]) {
            low = mid + 1;
        } else {
            high = mid;
        }
    }
    return a[low];
}
```

Why compare with a[high]? If a[mid] > a[high], minimum is strictly to the right. Otherwise mid may itself be minimum, so high=mid.

Minimum with duplicates

Minimum — duplicates allowed

```
static int findMinWithDuplicates(int[] a) {
    int low = 0, high = a.length - 1;
    while (low < high) {
        int mid = low + (high - low) / 2;
        if (a[mid] > a[high]) low = mid + 1;
        else if (a[mid] < a[high]) high = mid;
        else high--; // ambiguous equality
    }
    return a[low];
}
```

Distinct version O(log n); duplicates version worst-case O(n), e.g. all elements equal.

Rotation count

Agar array distinct values ke saath sorted array ka rotation hai, minimum element ka index hi right-rotation pivot / rotation count hota hai. Example [4,5,6,1,2,3] → min index 3, rotation count 3.

7. Peak Element, Mountain and Bitonic Arrays

Peak: index i jahan a[i] neighbours se greater ho. Boundary par missing neighbour ko -infinity samjha ja sakta hai. Problem ke exact definition ko follow karo.

Find any peak — O(log n)

```
static int findPeakElement(int[] a) {
    int low = 0, high = a.length - 1;

    while (low < high) {
        int mid = low + (high - low) / 2;
        if (a[mid] > a[mid + 1]) {
            high = mid;
        } else {
            low = mid + 1;
        }
    }
    return low;
}
```

Agar slope down ho rahi hai, peak mid ya uske left mein hogi. Agar a[mid] < a[mid+1], upward slope hai, toh peak right mein exist karti hai. Equal adjacent values ke liye assumptions alag ho sakti hain.

Mountain array

Mountain array strictly increase karti hai, ek peak, phir strictly decrease. Pehle peak index find karo. Target ko ascending half mein normal binary search aur descending half mein reverse comparator ke saath search karo.

Peak + ascending/descending binary search

```
static int peakIndex(int[] a) {
    int l = 0, r = a.length - 1;
    while (l < r) {
        int m = l + (r - l) / 2;
        if (a[m] < a[m + 1]) l = m + 1;
        else r = m;
    }
    return l;
}

static int ascSearch(int[] a, int l, int r, int x) {
    while (l <= r) {
        int m = l + (r-l)/2;
        if (a[m] == x) return m;
        if (a[m] < x) l = m+1; else r = m-1;
    }
    return -1;
}

static int descSearch(int[] a, int l, int r, int x) {
    while (l <= r) {
        int m = l + (r-l)/2;
        if (a[m] == x) return m;
        if (a[m] > x) l = m+1; else r = m-1;
    }
    return -1;
}

static int findInMountain(int[] a, int target) {
    int p = peakIndex(a);
    int left = ascSearch(a, 0, p, target);
    if (left != -1) return left;
    return descSearch(a, p+1, a.length-1, target);
}
```

Bitonic array

Bitonic array pehle strictly increasing, phir strictly decreasing hoti hai. Ismein peak search + do directional binary searches use karo. Complexity O(log n).

Important edge cases

- Single element is peak.
- Peak first/last index par ho sakti hai if definition allows boundary peak.
- Plateau (equal adjacent values) ke liye standard strict-peak template directly valid nahi.

8. Nearly Sorted and Infinite Sorted Arrays

Nearly sorted array

Nearly sorted ka common interview meaning: har element apni sorted position se at most 1 index door hai. Mid ke saath mid-1 aur mid+1 check karo. Agar target nahi, two positions skip karke interval update karo.

Search in array where each item is at most one position away

```
static int searchNearlySorted(int[] a, int target) {
    int low = 0, high = a.length - 1;
    while (low <= high) {
        int mid = low + (high-low)/2;

        if (a[mid] == target) return mid;
        if (mid-1 >= low && a[mid-1] == target) return mid-1;
        if (mid+1 <= high && a[mid+1] == target) return mid+1;

        if (a[mid] < target) low = mid + 2;
        else high = mid - 2;
    }
    return -1;
}
```

Search in infinite sorted array

Array ki length unknown ya access cost model mein effectively infinite hai. Pehle range exponentially expand karo: [0,1], [0,3], [0,7], [0,15]... Jab a[high] >= target ho, us bounded range mein binary search.

Exponential range + binary search

```
static int searchInfinite(int[] a, int target) {
    if (a.length == 0) return -1;
    if (a[0] == target) return 0;

    int low = 1, high = 1;
    while (high < a.length && a[high] < target) {
        low = high + 1;
        high = high * 2 + 1;
        if (high < 0) high = Integer.MAX_VALUE;
    }
    high = Math.min(high, a.length - 1);

    while (low <= high) {
        int mid = low + (high-low)/2;
        if (a[mid] == target) return mid;
        if (a[mid] < target) low = mid+1;
        else high = mid-1;
    }
    return -1;
}
```

Real Java array finite hota hai; `a.length` guard dikhaya gaya hai. True infinite random-access source mein out-of-range access ka behavior API define karega.

Complexity: target index p ho toh range expansion O(log p), binary search O(log p); total O(log p).

Exponential Search

Sorted array mein target ke aas-paas range find karne ke liye index 1,2,4,8,... double karo. Phir us interval mein binary search. Jab target start ke paas ho, useful ho sakta hai.

9. Binary Search on Answer — sabse important advanced pattern

Kabhi array mein target search nahi karna hota. Minimum capacity, minimum speed, maximum minimum distance jaise answer find karne hote hain. Candidate answer ko test karne wali condition monotonic honi chahiye.

Monotonic predicate

Pattern	Candidate x ke liye condition	Shape
Minimum feasible x	x chhota: false; x bada: true	FFFFTTTT
Maximum feasible x	x chhota: true; x bada: false	TTTTFFFF

Minimum feasible answer ke liye first true dhoondo. Maximum feasible answer ke liye last true dhoondo. Binary search ka main kaam transition point find karna hai.

First true — generic template

Minimum feasible answer

```
static int firstTrue(int low, int high) {
    // Assumption: predicate(low..high) is F...F,T...T
    while (low < high) {
        int mid = low + (high-low)/2;
        if (feasible(mid)) high = mid;
        else low = mid + 1;
    }
    return low;
}
```

Last true — generic template

Maximum feasible answer

```
static int lastTrue(int low, int high) {
    // Assumption: predicate is T...T,F...F
    while (low < high) {
        int mid = low + (high-low+1)/2; // upper mid
        if (feasible(mid)) low = mid;
        else high = mid - 1;
    }
    return low;
}
```

Last true mein upper mid important hai. Agar lower mid use karoge aur low=mid update hoga, low==high-1 par loop stuck ho sakta hai.

Answer-space problem solve karne ka 6-step framework

- 1. Answer ki minimum aur maximum possible bounds likho.
- 2. Ek candidate answer x assume karo.
- 3. `can(x)` / `feasible(x)` function define karo.
- 4. Prove karo predicate monotonic hai.
- 5. First true ya last true template choose karo.
- 6. Boundary cases aur returned answer ko verify karo.

Most common interview mistake: binary search ka code yaad hai, lekin `feasible(mid)` ka correct greedy/check function nahi bana paate. Problem ka main thinking aksar feasibility function mein hota hai.

Complexity

Agar answer range size R hai aur feasibility check O(n), total time O(n log R). Binary search ki complexity sirf O(log R) nahi; predicate check ka cost bhi multiply hota hai.

10. Aggressive Cows / Maximize minimum distance

Stalls ke positions diye hain. C cows place karni hain, minimum pairwise distance maximize karna hai. Candidate distance d feasible hai ya nahi, greedy se check karo: first stall par cow, phir har next stall par cow jab last placed cow se distance >= d.

Aggressive Cows — Java

```
import java.util.*;

static boolean canPlace(int[] stalls, int cows, int dist) {
    int placed = 1;
    int last = stalls[0];

    for (int i = 1; i < stalls.length; i++) {
        if (stalls[i] - last >= dist) {
            placed++;
            last = stalls[i];
            if (placed >= cows) return true;
        }
    }
    return false;
}

static int maxMinDistance(int[] positions, int cows) {
    if (cows < 2 || cows > positions.length)
        throw new IllegalArgumentException();

    int[] a = positions.clone();
    Arrays.sort(a);

    int low = 0;
    int high = a[a.length-1] - a[0];

    while (low < high) {
        int mid = low + (high-low+1)/2; // upper mid
        if (canPlace(a, cows, mid)) low = mid;
        else high = mid-1;
    }
    return low;
}
```

Dry run

Stalls [1,2,4,8,9], cows=3. Distance 3 feasible: place at 1,4,8. Distance 4? place 1,8, but 9 too close: only 2 cows. Answer 3.

Candidate distance	Greedy placement	Feasible?
2	1, 4, 8	Yes
3	1, 4, 8	Yes
4	1, 8	No
5	1, 8	No

Complexity: sorting O(n log n), each feasibility O(n), answer range O(log(max-min)); total O(n log n + n log R). Extra space depends on sorting implementation and cloned array O(n).

11. Allocate Books / Split Array Largest Sum

Books ke pages array mein hain. Contiguous books ko m students mein allocate karo; har student ko at least one book; maximum pages assigned to any student ko minimize karo. Candidate maximum limit X feasible hai agar greedy scan mein required students <= m.

Allocate Books — Java

```
static boolean canAllocate(int[] pages, int students, long limit) {
    int used = 1;
    long sum = 0;

    for (int p : pages) {
        if (p > limit) return false;
        if (sum + p <= limit) {
            sum += p;
        } else {
            used++;
            sum = p;
            if (used > students) return false;
        }
    }
    return true;
}

static long allocateBooks(int[] pages, int students) {
    if (pages.length == 0 || students <= 0 ||
        students > pages.length) return -1;

    long low = 0, high = 0;
    for (int p : pages) {
        low = Math.max(low, p);
        high += p;
```

```
    }

    while (low < high) {
        long mid = low + (high-low)/2;
        if (canAllocate(pages, students, mid)) high = mid;
        else low = mid+1;
    }
    return low;
}
```

Why bounds?

- Minimum possible maximum >= largest single book, because a book cannot be split.
- Maximum possible maximum = sum of all pages, if one student gets everything.
- Bigger limit makes allocation easier: predicate false...true, so first true.

Example

pages=[12,34,67,90], students=2. Optimal split [12,34,67] and [90] gives max 113; split [12,34] and [67,90] gives max 157. Minimum possible maximum = 113.

Complexity: O(n log(sumPages - maxPage + 1)); O(1) auxiliary space excluding input.

12. Ship Packages Within D Days

Weights ko order preserve karte hue D days mein ship karna hai. Minimum ship capacity find karo. Candidate capacity C feasible hai agar greedy packing days <= D.

Ship packages — Java

```
static boolean canShip(int[] weights, int days, long cap) {
    int usedDays = 1;
    long load = 0;

    for (int w : weights) {
        if (w > cap) return false;
        if (load + w <= cap) {
            load += w;
        } else {
            usedDays++;
            load = w;
        }
    }
    return usedDays <= days;
}

static long shipWithinDays(int[] weights, int days) {
    long low = 0, high = 0;
    for (int w : weights) {
        low = Math.max(low, w);
        high += w;
    }

    while (low < high) {
        long mid = low + (high-low)/2;
        if (canShip(weights, days, mid)) high = mid;
        else low = mid+1;
    }
    return low;
}
```

Similar family

- Minimum maximum load / capacity.
- Contiguous partition into at most k groups.
- Minimize largest segment sum.
- Painter's Partition Problem.
- Split Array Largest Sum.

In sab mein common structure: non-negative values, order preserve, candidate maximum bound, greedy feasibility, first true binary search.

13. Koko Eating Bananas — minimum speed

Koko ko piles diye hain aur H hours. Speed k bananas/hour par each pile ko ceil(pile/k) hours lagenge. Minimum k find karo jisse total hours <= H.

Koko — Java

```
static boolean canFinish(int[] piles, int h, int speed) {
    long hours = 0;
    for (int p : piles) {
        hours += (p + (long)speed - 1) / speed; // ceil(p/speed)
        if (hours > h) return false;
    }
    return true;
}

static int minEatingSpeed(int[] piles, int h) {
    int low = 1, high = 0;
    for (int p : piles) high = Math.max(high, p);

    while (low < high) {
        int mid = low + (high-low)/2;
        if (canFinish(piles, h, mid)) high = mid;
        else low = mid+1;
    }
    return low;
}
```

Ceil division formula for positive integers: `(p + k - 1) / k`. Cast to long before addition to avoid integer overflow.

More answer-space problems

Problem	Search answer	Feasibility idea
Smallest Divisor Given Threshold	Minimum divisor	Sum of ceil(a[i]/d) <= threshold
Minimum Days to Make Bouquets	Minimum days	Count consecutive bloomed flowers; bouquets possible?
Minimum Time to Complete Trips	Minimum time	Sum of time/t per machine >= trips
Maximum Candies per Child	Maximum piece size	Total pieces >= children
Maximum Number of Tasks	Maximum count	Check resource/time budget
Gas Station (advanced)	Minimize max gap	Check whether added stations can reduce all gaps

Smallest Divisor

Smallest divisor

```
static boolean divisorWorks(int[] a, int threshold, int d) {
    long sum = 0;
    for (int x : a) {
        sum += (x + (long)d - 1) / d;
        if (sum > threshold) return false;
    }
    return true;
}

static int smallestDivisor(int[] a, int threshold) {
    int low = 1, high = 0;
    for (int x : a) high = Math.max(high, x);

    while (low < high) {
        int mid = low + (high-low)/2;
        if (divisorWorks(a, threshold, mid)) high = mid;
        else low = mid+1;
    }
    return low;
}
```

14. Minimum Days to Make Bouquets

Bloom days array, m bouquets, each bouquet needs k adjacent flowers. Minimum day D find karo. Candidate D par bloomed flower means bloomDay[i] <= D. Greedy consecutive counter se bouquets count karo.

Bouquet feasibility + binary search

```
static boolean canMake(int[] bloom, int m, int k, int day) {
    int bouquets = 0, consecutive = 0;

    for (int d : bloom) {
        if (d <= day) {
            consecutive++;
            if (consecutive == k) {
                bouquets++;
                consecutive = 0;
            }
        } else {
            consecutive = 0;
        }
        if (bouquets >= m) return true;
    }
    return false;
}

static int minDays(int[] bloom, int m, int k) {
    if ((long)m * k > bloom.length) return -1;

    int low = Integer.MAX_VALUE, high = Integer.MIN_VALUE;
    for (int d : bloom) {
        low = Math.min(low, d);
        high = Math.max(high, d);
    }

    while (low < high) {
        int mid = low + (high-low)/2;
        if (canMake(bloom, m, k, mid)) high = mid;
        else low = mid+1;
    }
    return low;
}
```

Important: `(long)m*k` use karo; int multiplication overflow kar sakti hai. Bouquets banate waqt k consecutive flowers use hone ke baad counter reset hota hai.

15. Square Root and Nth Root

Integer square root

Floor square root: largest integer x such that x*x <= n. `x*x` int overflow kar sakta hai; use `(long)x*x`.

Floor square root — binary search

```
static int floorSqrt(int n) {
    if (n < 0) throw new IllegalArgumentException();

    int low = 0, high = n, ans = 0;
    while (low <= high) {
        int mid = low + (high-low)/2;
        long square = (long)mid * mid;

        if (square <= n) {
            ans = mid;
            low = mid+1;
        } else {
            high = mid-1;
        }
    }
    return ans;
}
```

Nth root using safe comparison

Nth root integer m find karo jahan m^n <= x but (m+1)^n > x. Multiplication overflow avoid karne ke liye multiply karte waqt x se compare karke early stop karo.

Compare power without overflow

```
static int comparePower(long base, int exp, long limit) {
    long result = 1;
    for (int i = 0; i < exp; i++) {
        if (base != 0 && result > limit / base) return 1;
        result *= base;
    }
    return Long.compare(result, limit);
}

static long floorNthRoot(long x, int n) {
    long low = 0, high = x, ans = 0;
    while (low <= high) {
        long mid = low + (high-low)/2;
        int cmp = comparePower(mid, n, x);
        if (cmp <= 0) {
```

```
            ans = mid;
            low = mid+1;
        } else {
            high = mid-1;
        }
    }
    return ans;
}
```

Floating-point binary search

Real answer chahiye, jaise sqrt(2) to fixed precision. Interval ko repeatedly half karo jab tak width epsilon se chhoti na ho.

Square root to epsilon precision

```
static double sqrtApprox(double x, double eps) {
    if (x < 0) throw new IllegalArgumentException();

    double low = 0;
    double high = Math.max(1.0, x);

    while (high - low > eps) {
        double mid = low + (high-low)/2.0;
        if (mid * mid <= x) low = mid;
        else high = mid;
    }
    return low;
}
```

Fixed 60 iterations bhi common approach hai. Floating-point problems mein termination `low < high` par depend mat karo; epsilon ya iteration count use karo.

16. Search in 2D Matrices

Matrix type A: flattened sorted matrix

Har row sorted hai aur next row ka first element previous row ke last element se bada hai. Matrix ko virtually sorted 1D array samajh sakte ho. Index mid ko row=mid/cols, col=mid%cols mein convert karo.

Search a row-wise flattened sorted matrix

```
static boolean searchMatrix(int[][] a, int target) {
    int rows = a.length;
    if (rows == 0) return false;
    int cols = a[0].length;
    if (cols == 0) return false;

    int low = 0, high = rows*cols - 1;
    while (low <= high) {
        int mid = low + (high-low)/2;
        int value = a[mid/cols][mid%cols];

        if (value == target) return true;
        if (value < target) low = mid+1;
        else high = mid-1;
    }
    return false;
}
```

Time O(log(rows*cols)); auxiliary O(1). Very large dimensions mein rows*cols int overflow risk check karo.

Matrix type B: rows and columns individually sorted

Har row left-to-right ascending aur har column top-to-bottom ascending, lekin flattened order globally sorted nahi. Top-right se staircase search O(rows+cols) hota hai. Binary search per row O(rows log cols) bhi possible.

Staircase search

```
static boolean searchRowColSorted(int[][] a, int target) {
    int rows = a.length;
    if (rows == 0) return false;
    int cols = a[0].length;
    if (cols == 0) return false;

    int r = 0, c = cols-1;
    while (r < rows && c >= 0) {
        if (a[r][c] == target) return true;
        if (a[r][c] > target) c--;
        else r++;
    }
    return false;
}
```

Which matrix approach?

Matrix property	Best common approach	Time
Globally sorted if flattened	1D virtual binary search	O(log(mn))
Each row sorted only	Binary search each row	O(m log n)
Rows + columns sorted	Staircase from top-right	O(m+n)
Rows + columns sorted, special constraints	Divide-and-conquer variants	Problem dependent

17. Median of Two Sorted Arrays

Do sorted arrays A and B ka combined median O(log(min(m,n))) mein find karna classic hard interview problem hai. Smaller array par binary search karo; partition dono arrays ko left/right halves mein divide karta hai.

Partition idea

- Total left partition size = (m+n+1)/2.
- i elements A se left; j = leftSize-i elements B se left.
- Valid partition condition: Aleft <= Bright and Bleft <= Aright.
- Valid partition par odd total: max(left); even total: average(max(left), min(right)).

Median of two sorted arrays — Java

```
static double findMedianSortedArrays(int[] A, int[] B) {
    if (A.length > B.length) return findMedianSortedArrays(B, A);

    int m = A.length, n = B.length;
    int low = 0, high = m;
    int leftSize = (m+n+1)/2;

    while (low <= high) {
        int i = low + (high-low)/2;
        int j = leftSize-i;

        int aL = (i == 0) ? Integer.MIN_VALUE : A[i-1];
        int aR = (i == m) ? Integer.MAX_VALUE : A[i];
        int bL = (j == 0) ? Integer.MIN_VALUE : B[j-1];
        int bR = (j == n) ? Integer.MAX_VALUE : B[j];

        if (aL <= bR && bL <= aR) {
            if ((m+n)%2 == 1) return Math.max(aL,bL);
            long leftMax = Math.max(aL,bL);
            long rightMin = Math.min(aR,bR);
            return (leftMax + rightMin) / 2.0;
        } else if (aL > bR) {
            high = i-1;
        } else {
            low = i+1;
        }
    }
    throw new IllegalArgumentException("Inputs must be sorted");
}
```

Complexity: O(log(min(m,n))) time, O(1) auxiliary space. Empty arrays supported as long as total length is non-zero.

Kth element of two sorted arrays

Same partition idea, but left partition size k. Valid partition par kth smallest = max(Aleft,Bleft).

Kth smallest in two sorted arrays (1-indexed k)

```
static int kthElement(int[] A, int[] B, int k) {
    if (A.length > B.length) return kthElement(B, A, k);
    int m=A.length, n=B.length;
    if (k < 1 || k > m+n) throw new IllegalArgumentException();

    int low = Math.max(0, k-n);
    int high = Math.min(k, m);

    while (low <= high) {
        int i = low+(high-low)/2;
        int j = k-i;

        int aL = i==0 ? Integer.MIN_VALUE : A[i-1];
        int aR = i==m ? Integer.MAX_VALUE : A[i];
        int bL = j==0 ? Integer.MIN_VALUE : B[j-1];
        int bR = j==n ? Integer.MAX_VALUE : B[j];

        if (aL<=bR && bL<=aR) return Math.max(aL,bL);
        if (aL>bR) high=i-1;
        else low=i+1;
    }
    throw new IllegalArgumentException("Invalid input");
}
```

18. Other Search Algorithms — know the difference

Exponential Search

Sorted array ke start se target tak range double karke identify karta hai, phir binary search. Time O(log p), where p target position; worst O(log n). Useful when target near beginning or unbounded-style search.

Ternary Search

Unimodal function/array mein maximum ya minimum find karne ke liye interval ko 3 parts mein divide karta hai. Integer unimodal array ke peak ke liye binary slope search usually simpler O(log n). Ternary search har iteration ~2/3 interval rakhta hai, O(log n) but more comparisons.

Ternary search on unimodal function (conceptual)

```
static double ternaryMax(double left, double right) {
    for (int it = 0; it < 100; it++) {
        double m1 = left + (right-left)/3.0;
        double m2 = right - (right-left)/3.0;

        if (f(m1) < f(m2)) left = m1;
        else right = m2;
    }
    return (left+right)/2.0;
}
// Define f(x) for the actual problem.
```

Interpolation Search

Sorted, roughly uniformly distributed numeric data mein estimated position calculate karta hai. Average O(log log n) under favorable uniform distribution; worst O(n). Binary search O(log n) worst-case reliable hai.

Interpolation search — guarded Java version

```
static int interpolationSearch(int[] a, int x) {
    int low=0, high=a.length-1;

    while (low<=high && x>=a[low] && x<=a[high]) {
        if (a[low] == a[high]) {
            return a[low] == x ? low : -1;
        }

        long numerator = (long)(x-a[low]) * (high-low);
        int pos = low + (int)(numerator / (a[high]-a[low]));

        if (a[pos] == x) return pos;
        if (a[pos] < x) low=pos+1;
        else high=pos-1;
    }
    return -1;
}
```

Interpolation formula: pos = low + ((x-a[low])*(high-low))/(a[high]-a[low]). Real production code mein numeric overflow aur value range carefully handle karo.

Jump Search

Sorted array ko blocks of size √n mein jump karke block identify, phir linear scan. Time O(√n), extra O(1). Binary search usually faster for random-access arrays.

Comparison table

Algorithm	Data requirement	Typical time	Key note
Linear	Any order	O(n)	Simplest
Binary	Sorted / monotonic	O(log n)	Worst-case reliable
Lower/Upper bound	Sorted	O(log n)	Boundary index
Exponential	Sorted	O(log p)	Range discovery
Ternary	Unimodal function	O(log n)	Two probes
Interpolation	Sorted, near-uniform	Avg O(log log n), worst O(n)	Distribution-sensitive
Jump	Sorted	O(√n)	Block scan

19. Java Binary Search Toolkit

Reusable helpers

Compact utility class

```
import java.util.*;

class BinarySearchUtils {
    static int lowerBound(int[] a, int x) {
        int l=0, r=a.length;
        while (l<r) {
            int m=l+(r-l)/2;
            if (a[m]<x) l=m+1;
            else r=m;
        }
        return l;
    }

    static int upperBound(int[] a, int x) {
        int l=0, r=a.length;
        while (l<r) {
            int m=l+(r-l)/2;
            if (a[m]<=x) l=m+1;
            else r=m;
        }
        return l;
    }

    static boolean contains(int[] a, int x) {
        int i=lowerBound(a,x);
        return i<a.length && a[i]==x;
    }

    static int frequency(int[] a, int x) {
        return upperBound(a,x)-lowerBound(a,x);
    }
}
```

Java Arrays.binarySearch

Java standard library `Arrays.binarySearch(a, key)` sorted array par binary search karta hai. Match milne par index return. Not found par insertion point encode karke `-(insertionPoint)-1` return karta hai.

Decode result

```
int result = Arrays.binarySearch(a, target);

if (result >= 0) {
    System.out.println("Found at " + result);
} else {
    int insertionPoint = -result - 1;
    System.out.println("Insert at " + insertionPoint);
}
```

Duplicates hon toh kaunsa matching index return hoga, is par rely mat karo. First occurrence ke liye lower bound implement karo.

Comparator-based search

Objects ya custom records ke liye `List` sort + comparator use kar sakte ho. Binary search se pehle sorting order aur comparator same hona chahiye.

Binary search on sorted list

```
List<String> names = new ArrayList<>(
    Arrays.asList("Aman", "Kiran", "Ravi", "Zoya")
);
int index = Collections.binarySearch(names, "Ravi");
// If absent, negative insertion point encoding.
```

Midpoint formulas

Need	Formula
Lower mid	low + (high-low)/2
Upper mid	low + (high-low+1)/2
Inclusive exact search	while (low <= high)
Half-open lower bound	while (low < high), high = n
First true	false -> low=mid+1; true -> high=mid
Last true	upper mid; true -> low=mid; false -> high=mid-1

Overflow traps

- Use `long` for sum of array, page totals, time totals, products and distances where bounds can exceed int.
- Use `mid = low + (high-low)/2`; for upper mid use long arithmetic if high-low may approach Integer.MAX_VALUE.
- For ceil division positive x,d: `(x + d - 1) / d` with long cast before addition, or `x/d + (x%d==0?0:1)`.

- For square: `(long)mid*mid`; casting after multiplication is too late.

20. Complexity Analysis — explain it correctly

Classic binary search

Each iteration O(1) work and halves the search space. Recurrence T(n)=T(n/2)+O(1) → O(log n).

Implementation	Time	Auxiliary space	Output space
Iterative exact search	O(log n) worst	O(1)	O(1)
Recursive exact search	O(log n)	O(log n) call stack	O(1)
Lower/upper bound	O(log n)	O(1)	O(1)
Rotated distinct search	O(log n)	O(1)	O(1)
Rotated duplicates	Worst O(n)	O(1)	O(1)
Answer search + O(n) check	O(n log R)	Usually O(1)	O(1)
Median of two arrays	O(log min(m,n))	O(1)	O(1)
Matrix flattened search	O(log(mn))	O(1)	O(1)

Time vs space vs auxiliary space

- Time complexity: input size ke relation mein operations ka growth.
- Space complexity: algorithm execution ke dauran memory growth; convention ke hisaab se input storage include/exclude clearly state karo.
- Auxiliary space: input store karne ki memory ko chhodkar algorithm ki extra working memory. Recursive call stack auxiliary space mein count hota hai.
- Output space: returned output ke liye memory; interview mein auxiliary space se alag mention karna useful hai.

Answer-space search complexity

Agar R possible integer answers hain aur feasibility check C(n) time leta hai, binary search iterations O(log R) hongi. Total O(C(n) log R). Example allocate books: C(n)=O(n), R≈sumPages, so O(n log(sumPages)).

How to justify O(log n) formally

After k iterations, interval length at most n/2^k. Stop when length <= 1. Therefore n/2^k <= 1 → 2^k >= n → k >= $\log_2 n$. Number of iterations is O(log n).

Interview answer template: “Each iteration discards at least half of the remaining search space, and does constant work, so time is O(log n). The iterative implementation uses O(1) auxiliary space; the recursive version uses O(log n) stack space.”

21. Common mistakes and debugging checklist

- Wrong loop condition: inclusive interval uses low <= high; half-open boundary search uses low < high.
- Wrong high initialization: exact search high=n-1; lower bound high=n.
- Wrong update: low=mid or high=mid can cause infinite loops unless interval style and midpoint are designed for it.
- Wrong equality handling: first occurrence must continue left; last occurrence must continue right.
- Using lower bound when upper bound is required, or vice versa.
- Assuming rotated array has a globally sorted half when duplicates make endpoints ambiguous.
- Not checking empty array / empty matrix / one-element input.
- Using int for potentially large sums, products, distances or total hours.
- Using `(low+high)/2` and risking overflow.
- Applying sliding/greedy feasibility where monotonicity is not proven.
- Answer range wrong: forgot lower bound or upper bound candidate.
- Not validating feasibility at boundary values.
- Returning low when template actually searches last true, or returning high incorrectly.
- Floating-point loop never ends because equality comparison is used.
- Assuming `Arrays.binarySearch` returns first duplicate.

Pre-submit test checklist

Test	Expected behavior
Empty input	No crash; contract-specific result
One element	Match / not match
Target below minimum	-1 or insertion index 0
Target above maximum	-1 or insertion index n
All elements equal	Boundary logic correct
Many duplicates	First/last/count correct
Integer extremes	No overflow
Rotated at pivot 0	Works like sorted array
Rotated at last pivot	Correct min and target
Answer at minimum bound	First feasible correctly found
Answer at maximum bound	Last feasible correctly found

Proof questions interviewer may ask

- Why can you discard the left half?
- Why is the predicate monotonic?
- Why does greedy feasibility produce the correct minimum number of groups?
- Why do you use upper mid in last-true search?
- What is the loop invariant?
- What happens with duplicates?

22. Interview Problem Bank — Foundation to Advanced

Problems ko order mein solve karo. Hints solution ka direction batate hain; pehle khud attempt karo. Platform labels common examples hain, exact constraints problem statement se verify karo.

Level	Problem / task	Pattern / hint
Basic	Binary Search	Exact search in sorted array
Basic	Search Insert Position	Lower bound
Basic	First and Last Position	Boundary search
Basic	Count Occurrences	Upper - lower
Basic	Floor and Ceil	Lower/upper bound
Basic	Square Root of X	Last true: x*x <= n
Basic	Search in 2D Matrix	Flattened index
Easy-Med	Find Peak Element	Slope direction
Easy-Med	Search in Rotated Sorted Array	Identify sorted half
Easy-Med	Find Minimum in Rotated Array	Compare mid with high
Easy-Med	Single Element in Sorted Array	Pair parity / index pattern
Easy-Med	Find Smallest Letter Greater Than Target	Circular upper bound
Medium	Search in Nearly Sorted Array	Check mid and neighbours
Medium	Search in Infinite Sorted Array	Exponential range
Medium	Find K Closest Elements	Boundary / answer window
Medium	Koko Eating Bananas	Minimum feasible speed
Medium	Capacity to Ship Packages	Minimum feasible capacity
Medium	Minimum Days for Bouquets	Minimum feasible day
Medium	Smallest Divisor Given Threshold	Minimum feasible divisor
Medium	Aggressive Cows	Maximum feasible distance
Medium	Allocate Books	Minimum maximum pages
Medium	Split Array Largest Sum	Greedy + first true
Medium	Find Rotation Count	Minimum index
Medium	Peak in Mountain Array	Peak search
Hard	Median of Two Sorted Arrays	Partition smaller array
Hard	Kth Element of Two Sorted Arrays	Partition size k
Hard	Find in Mountain Array	Peak + two searches
Hard	Median of Row-wise Sorted Matrix	Value-space search + count
Hard	Kth Smallest in Sorted Matrix	Value-space / heap
Hard	Minimize Max Distance to Gas Station	Floating-point answer search
Hard	Maximum Minimum Distance	Last true predicate
Hard	Nth Root of Integer	Power comparison without overflow
Hard	Search in Rotated Array with Duplicates	Ambiguous endpoints
Hard	First True Version	Monotonic predicate
Hard	Minimum Time to Complete Trips	Monotonic feasibility

Extra pattern prompts

- Find first index where a[i] >= x.
- Find first index where a[i] > x.
- Find last index where a[i] <= x.
- Find first true in a boolean array F...F,T...T.
- Find last true in T...T,F...F.
- Find smallest capacity to split array into at most k contiguous groups.
- Find maximum minimum distance among selected positions.
- Find minimum rate to complete jobs before deadline.
- Find median of a row-wise sorted matrix.
- Find kth smallest element in a sorted matrix.
- Find minimum in rotated array when duplicates exist.
- Find a peak in an unsorted array.

23. Revision Sheet — formulas you must remember

Concept	Formula / rule
Safe mid	low + (high-low)/2
Upper mid	low + (high-low+1)/2
Exact search	low=0, high=n-1; while low<=high
Lower bound	first i with a[i]>=x
Upper bound	first i with a[i]>x
Count duplicates	upperBound(x)-lowerBound(x)
First occurrence	lowerBound, then verify equality
Last occurrence	upperBound-1, then verify equality
Insertion index	lowerBound(x)
Strict predecessor	lowerBound(x)-1
Strict successor	upperBound(x)
Flatten matrix	row=mid/cols, col=mid%cols
Integer sqrt	largest x with (long)x*x<=n
Minimum feasible	first true: F...F,T...T
Maximum feasible	last true: T...T,F...F
Answer search time	O(checkCost * log(answerRange))
Binary search recurrence	T(n)=T(n/2)+O(1)=O(log n)

Recognition cheat sheet

Question wording	Think of
Find target in sorted array	Classic binary search
First / last / count / insertion	Lower & upper bound
Rotated sorted	Sorted-half binary search
Minimum in rotated	Compare mid and high
Peak / mountain	Slope-based binary search
Minimum possible capacity / speed / days	First feasible answer
Maximum possible minimum distance	Last feasible answer
Kth / median of two sorted arrays	Partition binary search
Sorted matrix flattened	Virtual 1D search
Unknown range / infinite sorted	Exponential + binary search
Real-valued optimum	Floating-point binary search

Final interview explanation

> “I will first define the search space and prove that it is ordered or that the feasibility predicate is monotonic. Then I will choose the correct interval convention, maintain the invariant, and discard only the half that cannot contain the answer. Each iteration halves the range, giving logarithmic iterations. I will also state the cost of the feasibility check, if any, and distinguish auxiliary space from recursion-stack space.”

Suggested practice order: standard search → lower/upper bound → first/last → rotated → peak → answer-space → matrix → median/kth. Har pattern ke 4–5 questions bina solution dekhe solve karo.

End of handbook. Binary Search ko ratne ke bajaye search space, monotonicity, invariant aur boundary logic samjho—yehi skill new interview questions solve karwati hai.